Nutrify: Desi Dish Recognition and Calorie Estimation

Deliverable 1 – Problem Definition and Data Acquisition

Muhammad Huzaifa (24L-3019)
Muhammad Abdullah Rasheed (24L-3002)
Artificial Intelligence, BSE-5B, FAST-NUCES Lahore

October 2026

1 Phase 1: Problem Definition

1.1 Problem Statement

Tracking what we eat is one of the simplest ways to manage weight and health, but it only works if logging a meal is quick. In South Asia it usually is not. Popular nutrition apps are built around Western food databases, so everyday dishes such as biryani, nihari, haleem and karahi are either missing, listed under unfamiliar names, or represented by a generic entry that does not match what is on the plate.

Users are left to search manually, guess which entry is closest, and estimate the portion by eye. The error is large because the calorie content of desi dishes depends heavily on the oil or ghee used, the cut of meat and the serving size, none of which a generic entry captures. Most people give up after a few days, which defeats the purpose of tracking.

Nutrify addresses this by turning a single photo of a meal into a dish name and a calorie range. The system has three parts: an image classification model that recognises 14 popular desi dishes, a calorie module that maps the recognised dish to a per-serving value with an adjustable portion size, and a web application with image upload and camera capture that returns the dish name, a confidence score and a calorie range in seconds.

1.2 Target Audience

Primary users. Everyday people in Pakistan and the wider South Asian region who want to keep an eye on what they eat, for example to lose or gain weight, follow a fitness plan, or simply eat more mindfully. They are comfortable taking a photo on a phone but do not want to type in meals or look up nutrition tables.

Secondary users. Students, fitness coaches and dietitians who need a quick estimate when discussing local food, and who currently rely on printed tables or Western-centric apps.

Research gap. Widely used food recognition benchmarks such as Food-101 are dominated by Western dishes, and there are few public, labelled image collections for Pakistani food. As a result, models trained on those benchmarks do not transfer well to dishes like nihari or haleem. This project helps fill that gap by assembling a curated dataset of desi dishes, documenting how it was collected and cleaned, and evaluating how the model behaves on look-alike dishes, different lighting and restaurant versus home-cooked photos.

1.3 AI Justification

Why not rule-based programming. Recognising a dish from a photo cannot be written as a set of if–else rules. The same dish looks different from one cook, restaurant or camera to the next, and the lighting, plating and background change from photo to photo. Dishes such as

nihari, haleem and aloo gosht are also visually similar brown stews, so simple colour or texture rules would confuse them. Any hand-written rule set would need constant exceptions and would still fail on photos it has not seen.

Why deep learning fits. Convolutional neural networks learn the relevant visual features, from edges and textures up to whole-dish patterns, directly from labelled examples instead of relying on features chosen by hand. This is the standard approach for image classification and it handles the variation in appearance that defeats fixed rules.

Why transfer learning. Collecting thousands of images per dish is not realistic for a student project. Starting from a network pretrained on a large general image collection (EfficientNet-B0) lets us reuse features it has already learned and fine-tune it on roughly 150 to 300 images per dish. We will compare it against a baseline CNN trained from scratch to show the benefit. EfficientNet-B0 is also small enough to serve through a web application.

Where AI stops. Machine learning is used only for the part that needs it, recognising the dish. The calorie value comes from a verified per-serving entry in a food composition database, scaled by the chosen portion size. Keeping that step deterministic makes the numbers traceable and easier to defend than a model that guesses calories directly.

2 Phase 2: Data Acquisition and Preprocessing

2.1 Collection

Raw images were collected from two sources, giving 5,013 images across the 14 dishes before any cleaning. Kaggle contributes 1,154 of them and Bing scraping the remaining 3,859.

Bing image scraping. The script `collect_images.py` downloads images from Bing using the `icrawler` package, with five query variants per dish. Every downloaded URL is logged in `raw_metadata.csv`, so each image can be traced back to where it came from.

Kaggle dataset. The script `import_dataset.py` imports four classes from the Kaggle "Indian Food Classification" dataset (Pushkar Jain) and maps them onto our dishes: `samosa` → samosa (262 images), `pakode` → pakora (278), `dal_makhani` → daal (321) and `kaathi_rolls` → paratha_roll (293). The remaining ten dishes come only from Bing.

Dish	Raw images	Source
daal	636	Bing + Kaggle (dal makhani, 321)
paratha_roll	566	Bing + Kaggle (kathi rolls, 293)
samosa	556	Bing + Kaggle (samosa, 262)
pakora	525	Bing + Kaggle (pakode, 278)
seekh_kebab	317	Bing
halwa_puri	314	Bing
chicken_biryani	296	Bing
nihari	291	Bing
chicken_tikka	289	Bing
pulao	261	Bing
haleem	254	Bing
aloo_gosht	242	Bing
palak_paneer	237	Bing
chicken_karahi	229	Bing
Total	**5,013**	

Table 1: Raw images per dish and their source, before cleaning.

2.2 Cleaning Implementation

Cleaning is done by `clean_data.py`, which never deletes the raw files: the images that pass are copied into a separate `data/clean` folder, together with `clean_metadata.csv` and `cleaning_report.csv`. The script applies the following checks:

- **Missing labels and duplicate metadata rows:** rows with no dish label, or repeated rows, are dropped.
- **Missing or corrupt files:** images that are not on disk or cannot be opened are dropped.
- **Tiny images:** images whose shorter side is below 150 pixels are dropped.
- **Extreme aspect ratio:** images with an aspect ratio above 3 are dropped.
- **Blur:** images whose Laplacian variance is below 40 are dropped as blurry.
- **Exact duplicates:** images with an identical MD5 hash are dropped.
- **Near duplicates:** images whose perceptual hash distance to an earlier image is at most 4 are dropped.

The script also records the width, height, brightness and blur score of every kept image, which are used for the exploratory analysis. Of the 5,013 raw images, 1,124 (22.4%) were removed and 3,889 (77.6%) were kept. Table 2 shows what each step removed, as reported in `cleaning_report.csv`; the report does not list separate counts for the missing-file, corrupt-file and tiny-image checks.

Cleaning step	**Images removed**
Missing label	0
Duplicate metadata rows	0
Exact duplicates (MD5)	697
Blurry (Laplacian variance < 40)	217
Near duplicates (perceptual hash ≤ 4)	205
Extreme aspect ratio (> 3)	5
Total removed	**1,124**
Images kept	**3,889**

Table 2: Images removed by each cleaning step (from `cleaning_report.csv`).

Table 3 gives the effect of cleaning on each dish. A visual spot check showed that most cleaned images are representative of their dish and not blurred, although some watermarked, text-overlaid or off-topic images survive (see the limitations below).

Dish	Raw	Kept after cleaning
samosa	556	498
pakora	525	482
daal	636	461
paratha_roll	566	422
chicken_biryani	296	248
seekh_kebab	317	241
nihari	291	236
haleem	254	227
chicken_tikka	289	224
pulao	261	197
palak_paneer	237	187
halwa_puri	314	171
aloo_gosht	242	167
chicken_karahi	229	128
Total	**5,013**	**3,889**

Table 3: Images per dish before and after cleaning.

2.3 Feature Strategy

The script `preprocess.py` prepares the cleaned images for model training in the following way.

Label encoding. The 14 dish names are sorted alphabetically and mapped to integer ids: aloo_gosht = 0, chicken_biryani = 1, chicken_karahi = 2, chicken_tikka = 3, daal = 4, haleem = 5, halwa_puri = 6, nihari = 7, pakora = 8, palak_paneer = 9, paratha_roll = 10, pulao = 11, samosa = 12 and seekh_kebab = 13. The mapping is saved in `classes.json`.

Resizing and scaling. Every image is resized to 224×224 pixels, the input size of EfficientNet-B0, and its pixel values are scaled to the range $[0, 1]$.

Stratified split. The 3,889 cleaned images are divided 70/15/15 into training, validation and test sets. The split is stratified by dish, so each set keeps roughly the same class proportions, and it uses a fixed random seed (42) so it can be reproduced. Because duplicates were removed before splitting, near-identical images cannot end up in both the training and test sets.

Split	Images
Train	2,722
Validation	583
Test	584
Total	**3,889**

Table 4: Size of each split.

Normalisation. The per-channel mean and standard deviation are computed from the training set only, so no information from the validation or test images leaks into preprocessing. They are saved in `norm_stats.json` and will be used to normalise the inputs during model training.

Channel	Mean	Std
Red	0.5901	0.2872
Green	0.4925	0.2861
Blue	0.3718	0.3095

Table 5: Channel statistics of the training set (pixel values in $[0, 1]$).

2.4 Exploratory Data Analysis

The charts below were generated by `eda.py` from the cleaned and raw metadata.

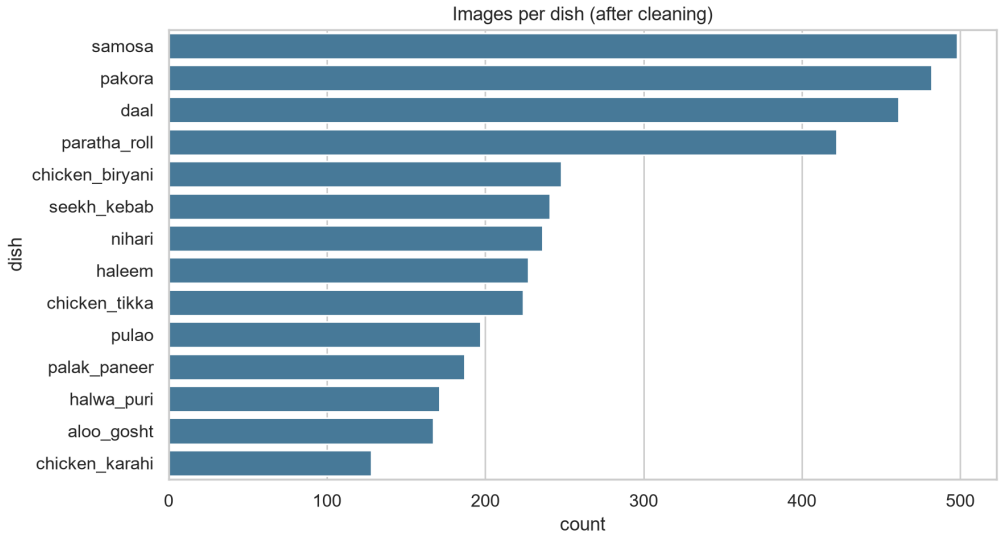


Figure 1: Images per dish after cleaning.

The classes are imbalanced. Samosa has the most images (498) and chicken_karahi the fewest (128), a ratio of about 3.9 to 1. The four largest classes (samosa, pakora, daal and paratha_roll) are exactly the four dishes that also draw on the Kaggle dataset.

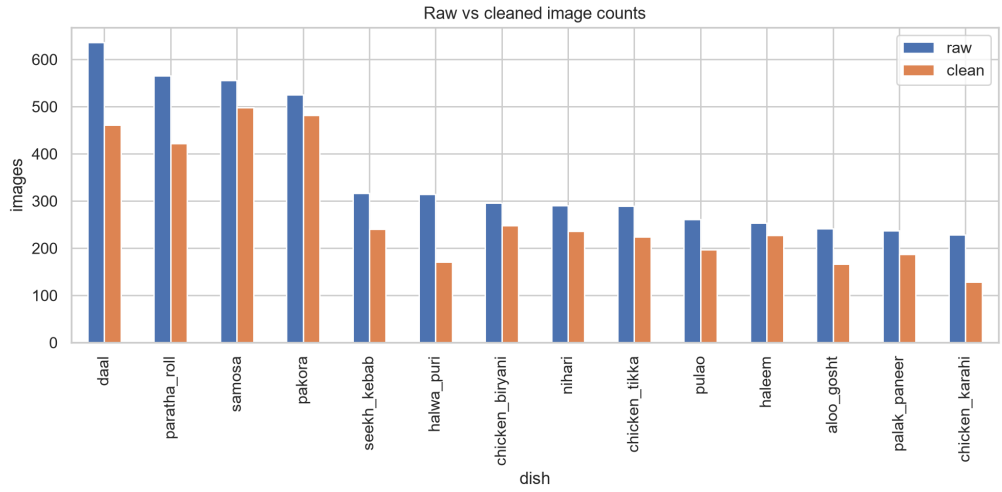


Figure 2: Raw versus cleaned image counts.

Cleaning reduced the collection from 5,013 to 3,889 images, and every dish lost some images.

The largest share was lost by halwa_puri (143 of 314, about 46%) followed by chicken_karahi (101 of 229, about 44%), while pakora lost the smallest share (43 of 525, about 8%).

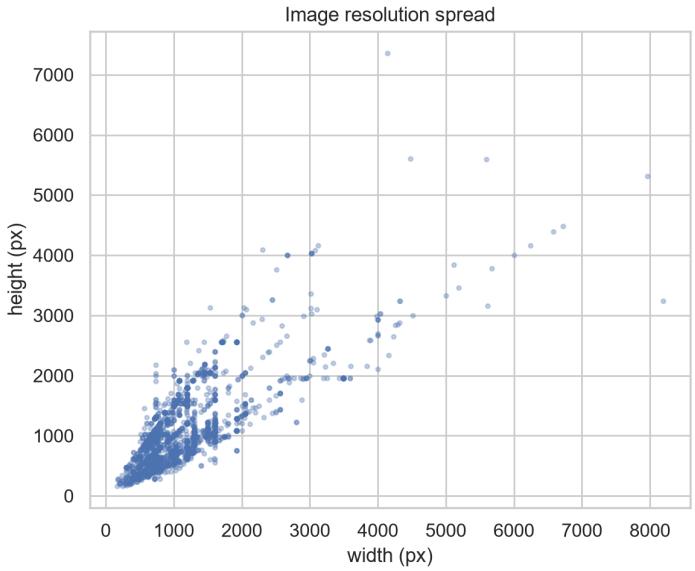


Figure 3: Image resolution spread.

Image resolution varies widely. Most images are smaller than about 2,000 pixels on each side, with a dense cluster below 1,500 pixels, but a small number exceed 4,000 pixels and the largest are around 8,000 pixels wide. Width and height grow together. This spread motivates resizing everything to a common 224×224 input.

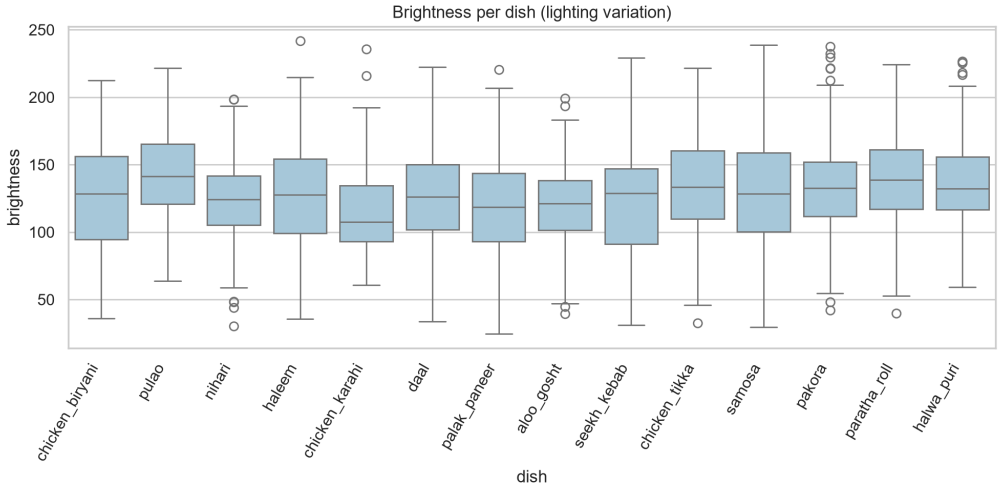


Figure 4: Brightness per dish (lighting variation).

Lighting varies a lot within every dish: for most dishes the whiskers extend from roughly 60 or lower to roughly 190 or higher, so each dish contains both very dark and very bright photos. The median brightness is fairly similar across dishes, from about 107 for chicken_karahi (darkest) to about 141 for pulao (brightest), so lighting differs more between individual images than between dishes.

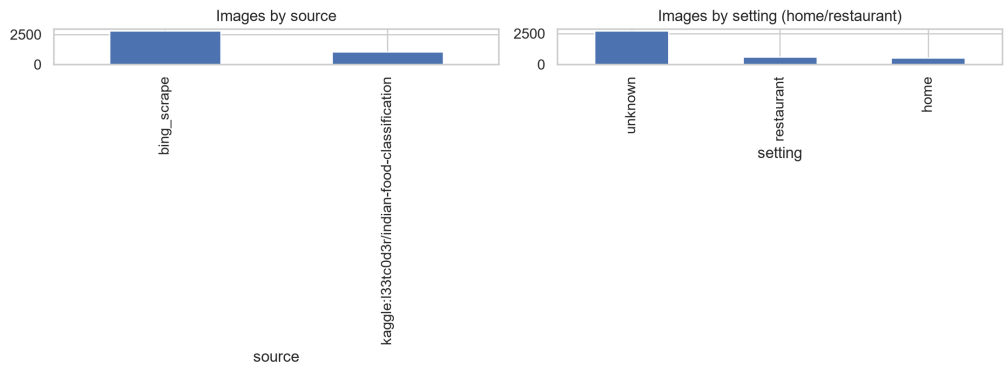


Figure 5: Images by source and by setting (home/restaurant).

Roughly 70% of the cleaned images come from Bing and roughly 30% from the Kaggle dataset. The setting is unknown for most images (roughly two thirds); among the rest, restaurant and home photos are about equally common.

aloo_gosht







chicken_biryani






chicken_karahi







chicken_tikka

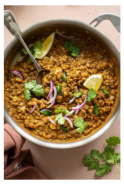




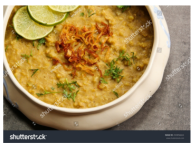


daal




haleem





halwa_puri







nihari







pakora







palak_paneer






The sample grid shows four random cleaned images per dish. The images vary widely in angle, plating and background, and most show the correct dish. The grid also exposes the problems listed under limitations: stock-photo watermarks (for example Alamy and Shutterstock), text banners and overlays, a menu flyer among the haleem images, and a few images that do not appear to show the labelled dish (for example in palak_paneer and halwa_puri).

2.5 Limitations

- **Watermarks and text overlays:** images scraped from Bing can contain watermarks, captions or text overlays, which the automatic blur and duplicate filters do not remove. Stock-photo watermarks (for example Alamy and Shutterstock), text banners (some in Urdu) and even a menu flyer are visible in the sample grid. A model could pick up on these instead of the food.

- **Label noise:** in the random sample grid (four images per dish), a few images do not appear to show the labelled dish (for example in palak_paneer and halwa_puri). The cleaning filters cannot detect wrong-dish images and the dataset was reviewed only by sampling, so some mislabelled images probably remain.

- **Source bias:** four dishes (samosa, pakora, daal and paratha_roll) draw on a single Kaggle dataset, while the other ten come only from Bing. The two sources may differ in photographic style, so a model could partly learn the source instead of the dish.

- **Close substitutes:** the Kaggle classes `dal_makhani` and `kaathi_rolls` stand in for daal and paratha_roll. They are similar dishes but not identical to what the project is meant to recognise.

- **Class imbalance:** raw counts range from 229 to 636 images per dish and cleaned counts from 128 to 498. chicken_karahi, with 128 images, is below the 150-image target we set, and with a 15% split its validation and test sets contain only about 19 images each, so its results will be noisy.

- **Licensing:** the scraped images belong to their original owners. They were collected for academic use only and are not redistributed: the dataset is excluded from the repository.